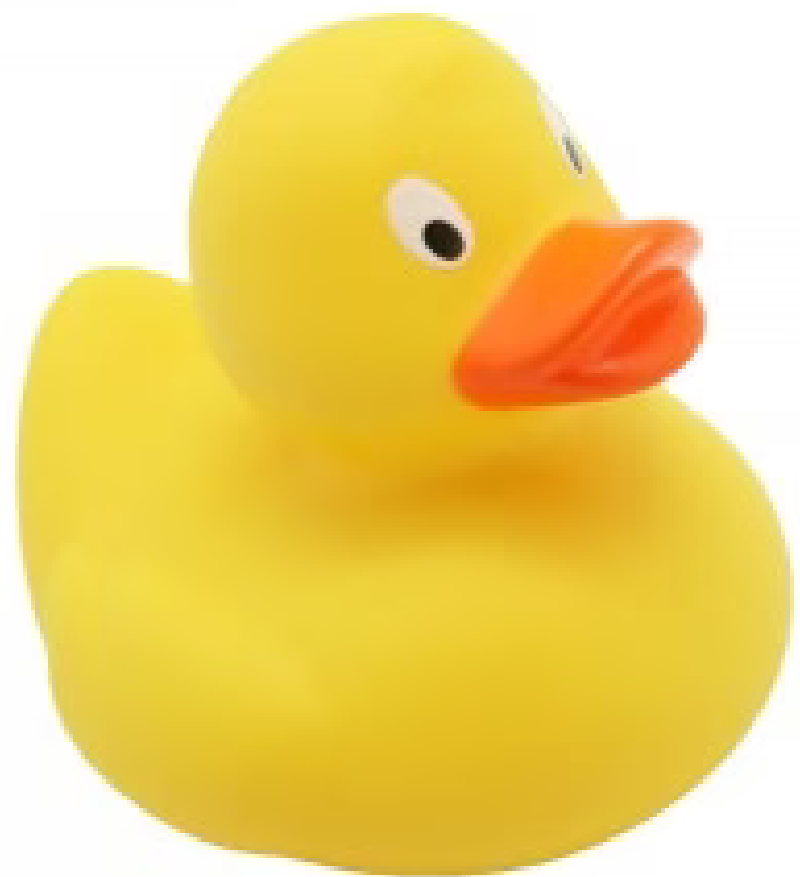


Do we know 'this'?



Call, Apply and Bind

How to understand the context of 'this' in JavaScript?

Pronoun "I" in English



Depends
on who
is
speaking

VS

Pronoun "This" in JS

Depends on
execution
context

JS

Global Context



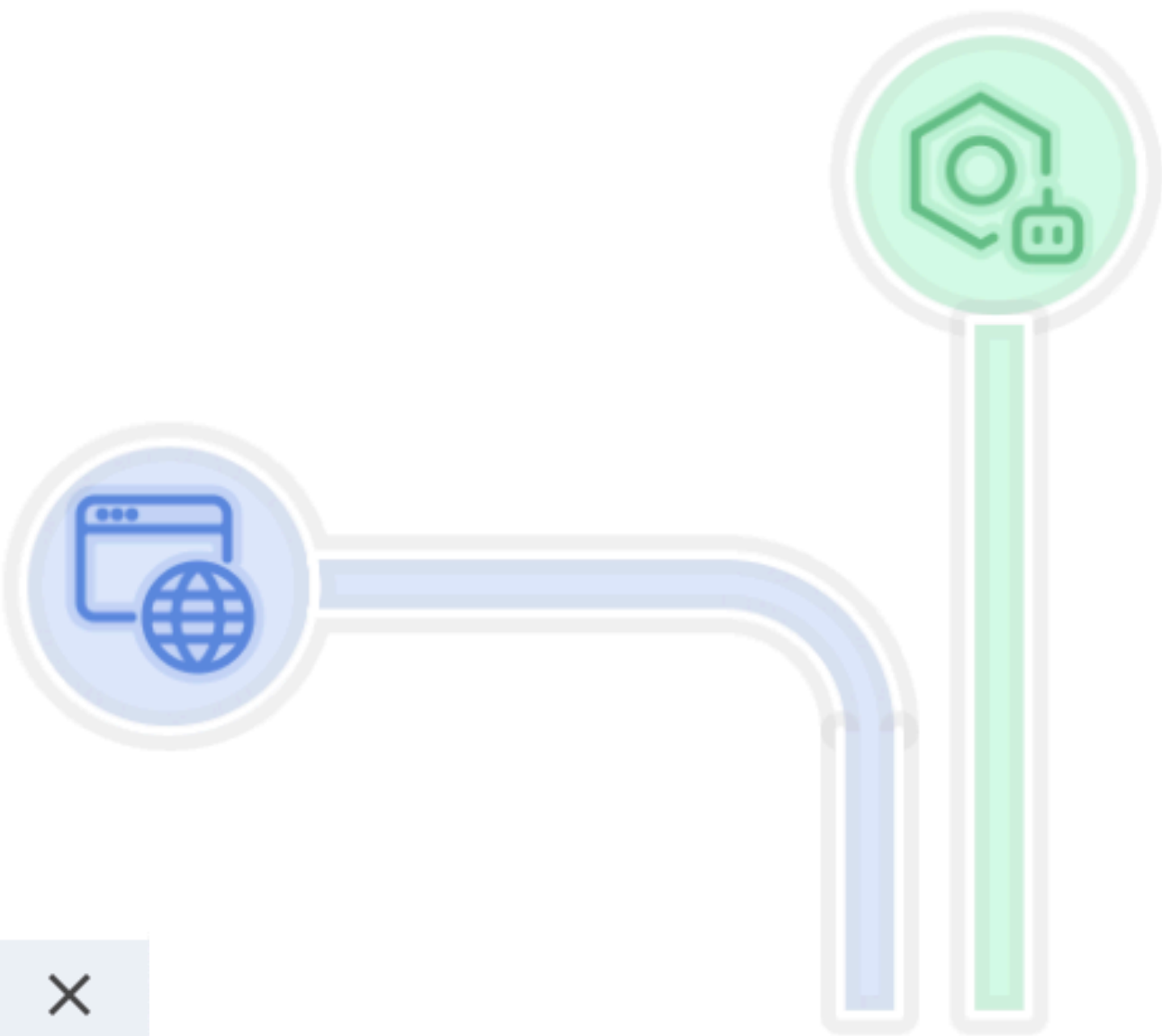
Node.js Context

'this' refers to the exports of the current module, returning an empty object.

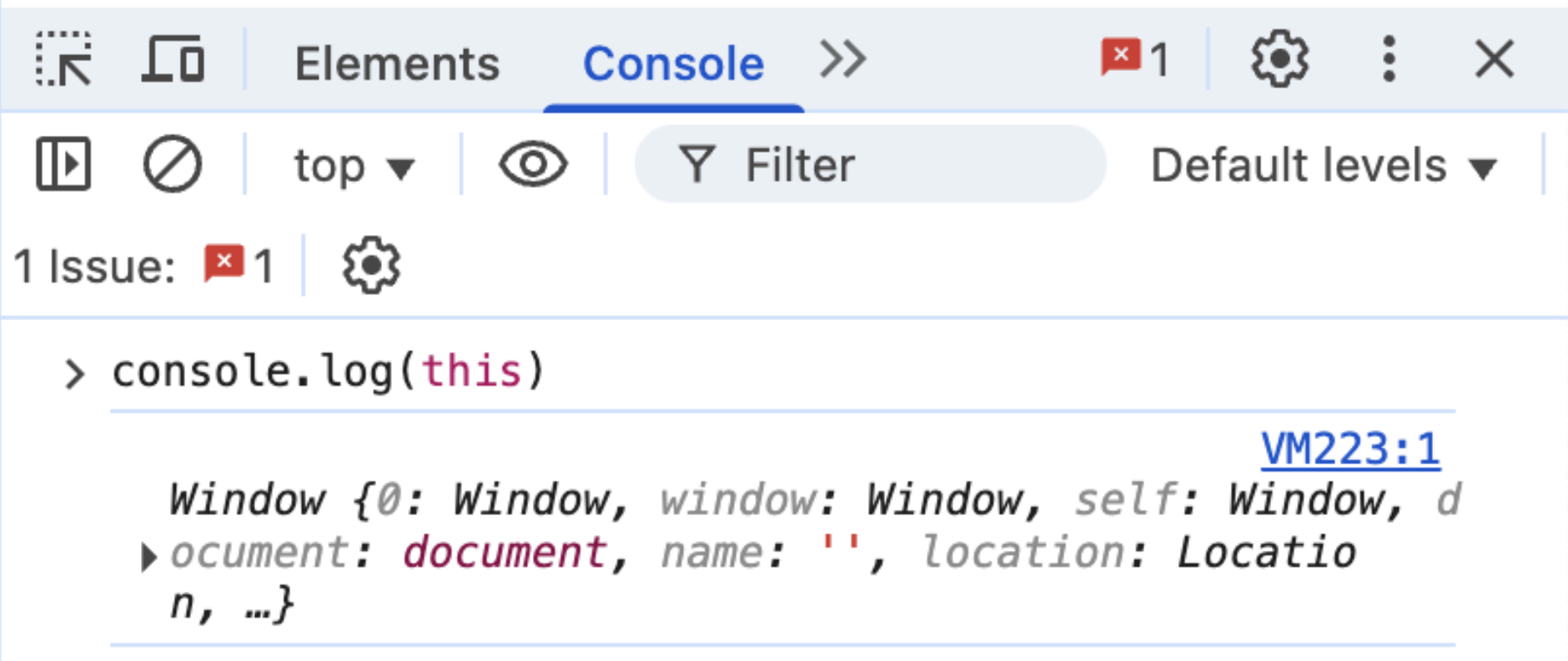


Browser Context

'this' refers to the window object, specific to browser execution.



```
JS script.js > [e] default > 🔑 title
1  exports.default = {
2    title: "Phone"
3  }
4  console.log(this);
```



```
PORTS  OUTPUT  PROBLEMS  TERMINAL  ...
● (base) → code node script.js
  {}
● (base) → code node script.js
  { default: { title: 'Phone' } }
○ (base) → code
```


Function Context

Browser

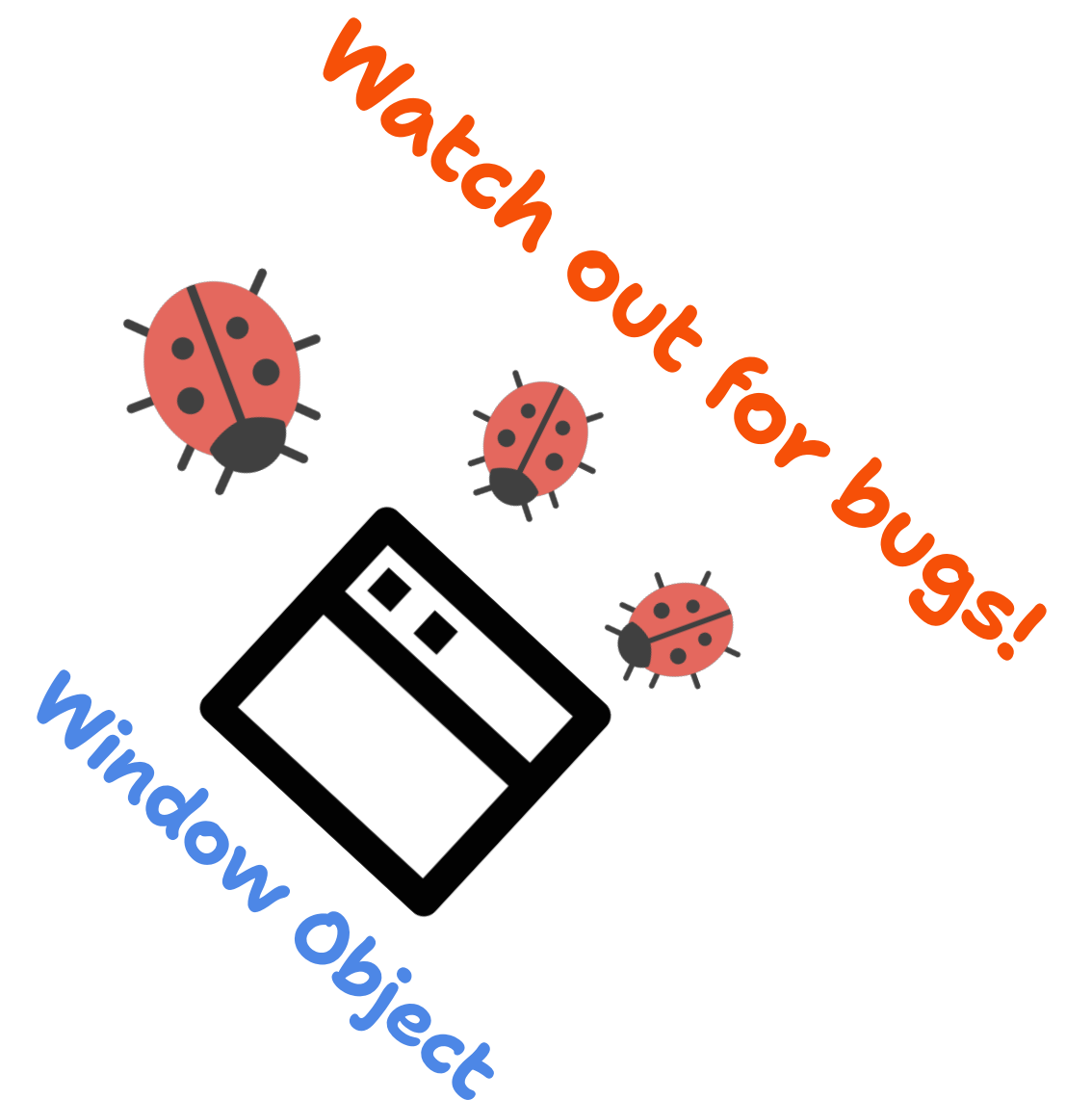
this in function invocation

```
window;  
function myFunc() {  
  this;  
}  
myFunc();
```

In a function invocation, the execution context is the global object.

At the time myFunc() is called, JavaScript automatically sets this as the global object.

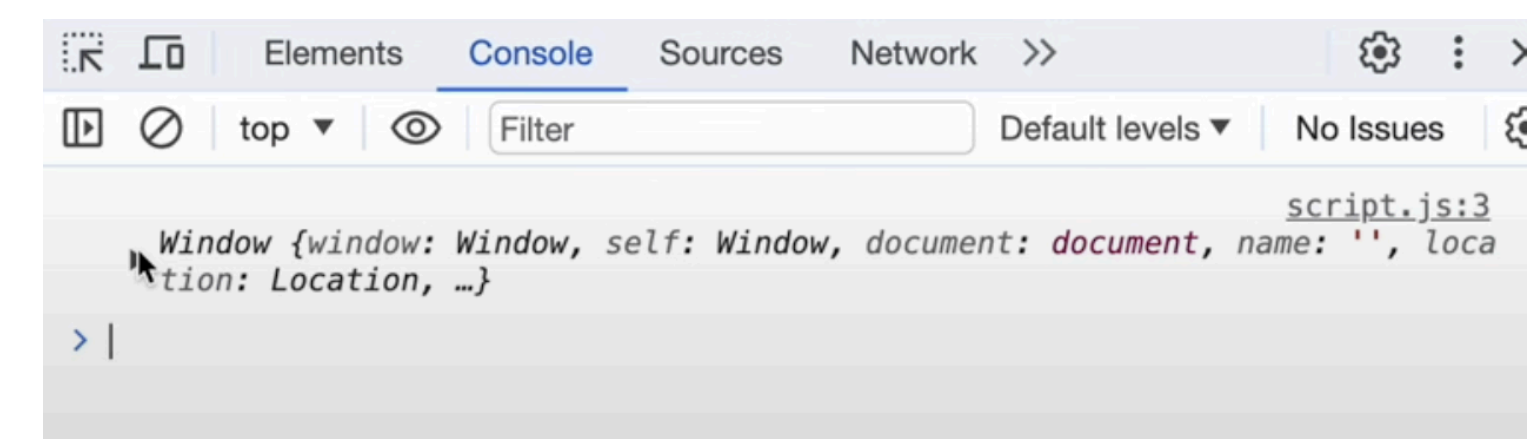
This is not assigned a value, until the function is invoked



```
1 function myFunction1() {  
2   function myFunction() {  
3     console.log(this);  
4   }  
5   myFunction();  
6 }  
7   
8 myFunction1();  
9
```

Even if a function is inside another function.

Output

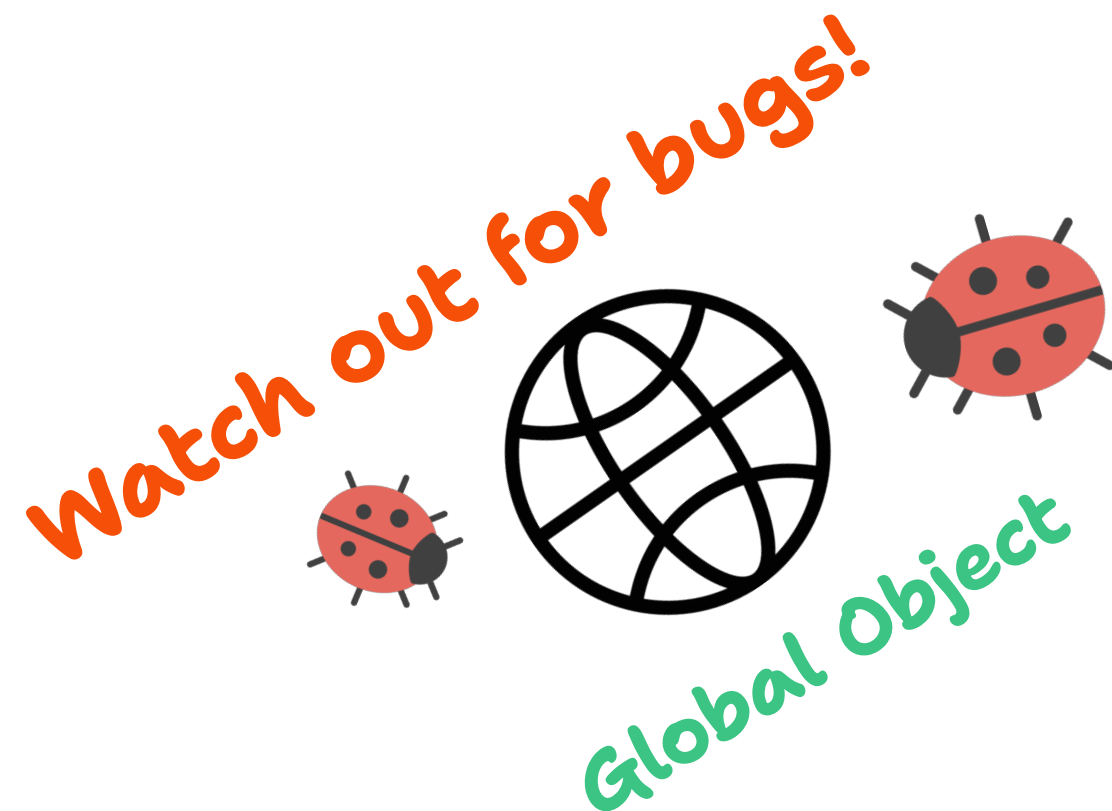


Function Context

Node.js

Node.js does not have a window object, as it does not have a user interface.

It has a global object, that includes Node-specific globals, timers, console.



```
JS script.js > myFunction
1 function myFunction() {
2   console.log(this);
3 }
4 myFunction();
5

PORTS OUTPUT PROBLEMS TERMINAL ...
zsh + - [ ] [ ] ... ^ X

• (base) → code node script.js
{ default: { title: 'Phone' } }
• (base) → code node script.js
<ref *1> Object [global] {
  global: [Circular *1],
  queueMicrotask: [Function: queueMicrotask],
  clearImmediate: [Function: clearImmediate],
  setImmediate: [Function: setImmediate] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  structuredClone: [Function: structuredClone],
  clearInterval: [Function: clearInterval],
  clearTimeout: [Function: clearTimeout],
  setInterval: [Function: setInterval],
  setTimeout: [Function: setTimeout] {
    [Symbol(nodejs.util.promisify.custom)]: [Getter]
  },
  atob: [Function: atob],
  btoa: [Function: btoa],
  performance: Performance {
    nodeTiming: PerformanceNodeTiming {
      name: 'node',
      entryType: 'node',
      startTime: 0,
      duration: 27.08437442779541,
      nodeStart: 3.3744592666625977,
      v8Start: 6.070124626159668,
      bootstrapComplete: 20.70958423614502,
      environment: 11.928417205810547,
      loopStart: -1,
      loopExit: -1,
      idleTime: 0
    },
    timeOrigin: 1700667535975.862
  },
  ...
}
```

Function Context

'use strict' directive



Both for Browser and Node.js, this is UNDEFINED in a function invocation in strict mode.

**this in function invocation,
strict mode**

```
undefined;  
function myFunc() {  
  'use strict';  
  this;  
}  
myFunc();
```

JavaScript does not automatically bind this to the global object, which is safer.

```
function multiply(a, b) {  
  'use strict'; // enable the strict mode  
  console.log(this === undefined); // => true  
  return a * b;  
}  
// multiply() function invocation with strict mode enabled  
// this in multiply() is undefined  
multiply(2, 5); // => 10
```

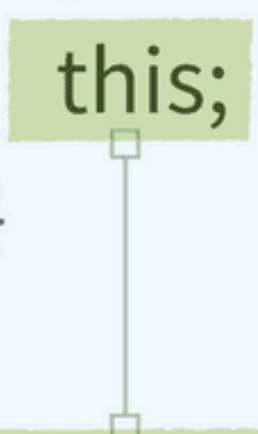
When multiply(2, 5) is invoked, this is undefined

Method Invocation Context

This is the object that owns the method in a method invocation

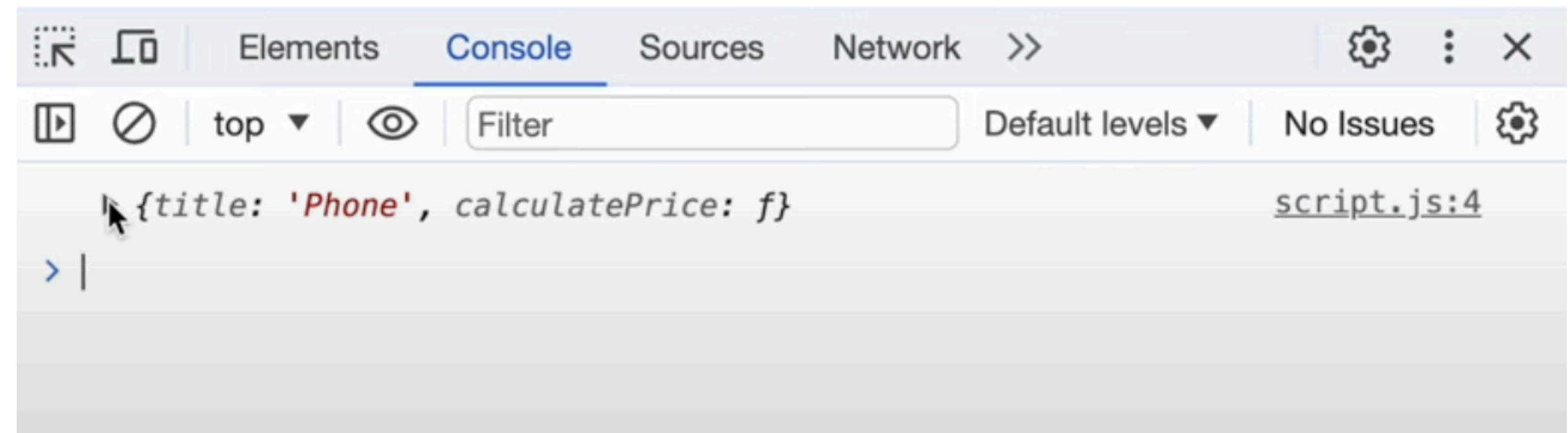
this in method invocation

```
var myObject = {  
  myMethod: function() {  
    this;  
  }  
};  
myObject.myMethod();
```

A diagram illustrating the 'this' keyword's context. A green box highlights 'this;' inside the 'myMethod' function. A vertical line with a small square at the bottom connects this box to another green box labeled 'myObject.myMethod()' in the line of code below the function definition.

```
JS script.js > ...  
1  const item = {  
2    title: "Phone",  
3    calculatePrice() {  
4      console.log(this);  
5    }  
6  }  
7    
8  item.calculatePrice()
```

Output

A screenshot of a web browser's developer console. The 'Console' tab is selected. It shows a single log entry: an object {title: 'Phone', calculatePrice: f}. The text 'script.js:4' is visible to the right of the object. Below the log entry, there is a prompt '> |' on a new line.

```
{title: 'Phone', calculatePrice: f} script.js:4  
> |
```

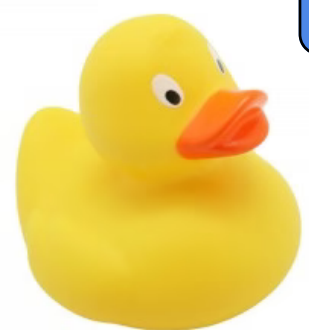
Constructor Invocation

this in constructor invocation

```
function Constructor() {  
  this;  
}  
var object = new Constructor();
```



Don't forget
the 'new'
operator!



This is the newly created object in a
constructor invocation

```
function Foo () {  
  // this is fooInstance  
  this.property = 'Default Value';  
}  
  
// Constructor invocation  
const fooInstance = new Foo();  
fooInstance.property; // => 'Default Value'
```


Arrow Function

An arrow function does not have its own execution context. It takes this from the outer function. If there is none, the context is the global object.

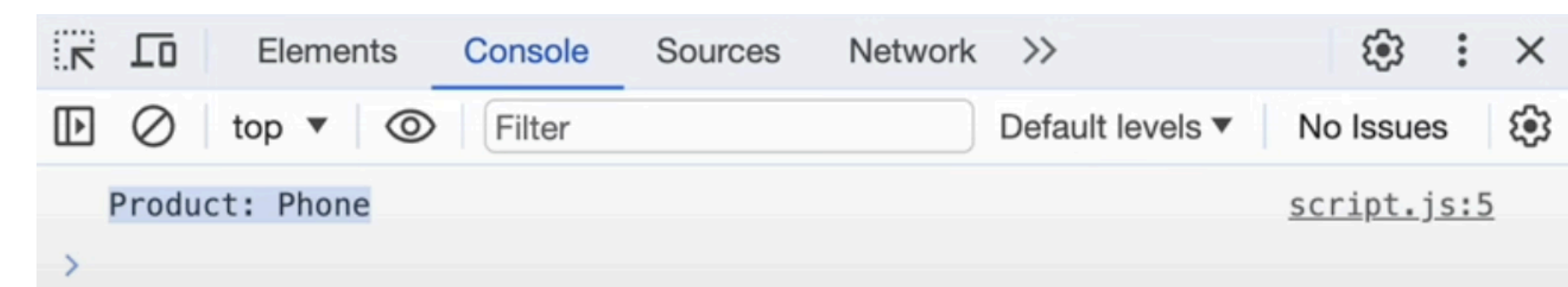
this in arrow function invocation

```
this;  
var myArrowFunc = () => {  
  this;  
};  
myArrowFunc();
```

```
JS script.js > [0] item > logTitle  
1  const item = {  
2    title: "Phone",  
3    logTitle: function () {  
4      setTimeout(() => {  
5        console.log(`Product: ${this.title}`);  
6      })  
7    }  
8  }  
9  
10 item.logTitle()
```

Arrow function inside
setTimeout does not have
its own this, but takes it
from the method on line 3.

Output



Difference

Same example with a function expression inside setTimeout:

```
JS script.js > ...
1  const item = {
2    title: "Phone",
3    logTitle: function () {
4      setTimeout(function () {
5        console.log(`Product: ${this.title}`);
6      })
7    }
8  }
9
10 item.logTitle()
```

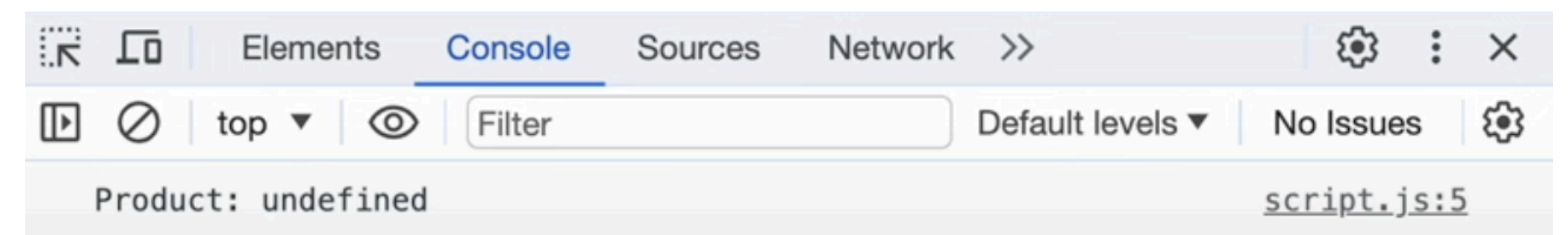
Just use
.bind(this)
on function
expression



Inside setTimeout function is executed after the main script, which means this inside setTimeout refers to the global window object.

Global window object does not have a property 'title'.

Output



Functions as Objects

Methods

Functions can have methods like any other object.



Properties

Functions can have properties like any other object.

Apply, Call and Bind



Apply

Call



Bind

Indirect Invocation

this is the first argument of `.call()` or `.apply()` in an indirect invocation

```
object.objectMethod.call( objectInstance, arguments )
```

```
JS script.js > ...
1  function myFunction() {
2    console.log(this);
3  }
4  const item = {title: "phone"}
5  myFunction.call(item);
6
```

```
object.objectMethod.apply(objectInstance, arrayOfArguments)
```

```
JS script.js  X  <> index.html
JS script.js > ...
1  function myFunction() {
2    console.log(this);
3  }
4  const item = {title: "phone"}
5  myFunction.apply(item);
6
```

Output



Bound function

this is the first argument of `myFunc.bind(thisArg)` when invoking a bound function

```
JS script.js > ...
1  function calcDiscount(age) {
2      if (age > 65) {
3          console.log(this.price / 2);
4      } else {
5          console.log(this.price);
6      }
7  }
8  const item = {title: "phone", price: 1000}
9  const calcDiscountForElderly = calcDiscount.bind(item, 70);
10 calcDiscountForElderly()
11
```

Contrary to `.apply()` and `.call()`, which invoke the function right away, the `.bind()` method only returns a new function supposed to be invoked later with a pre-defined `this` value.

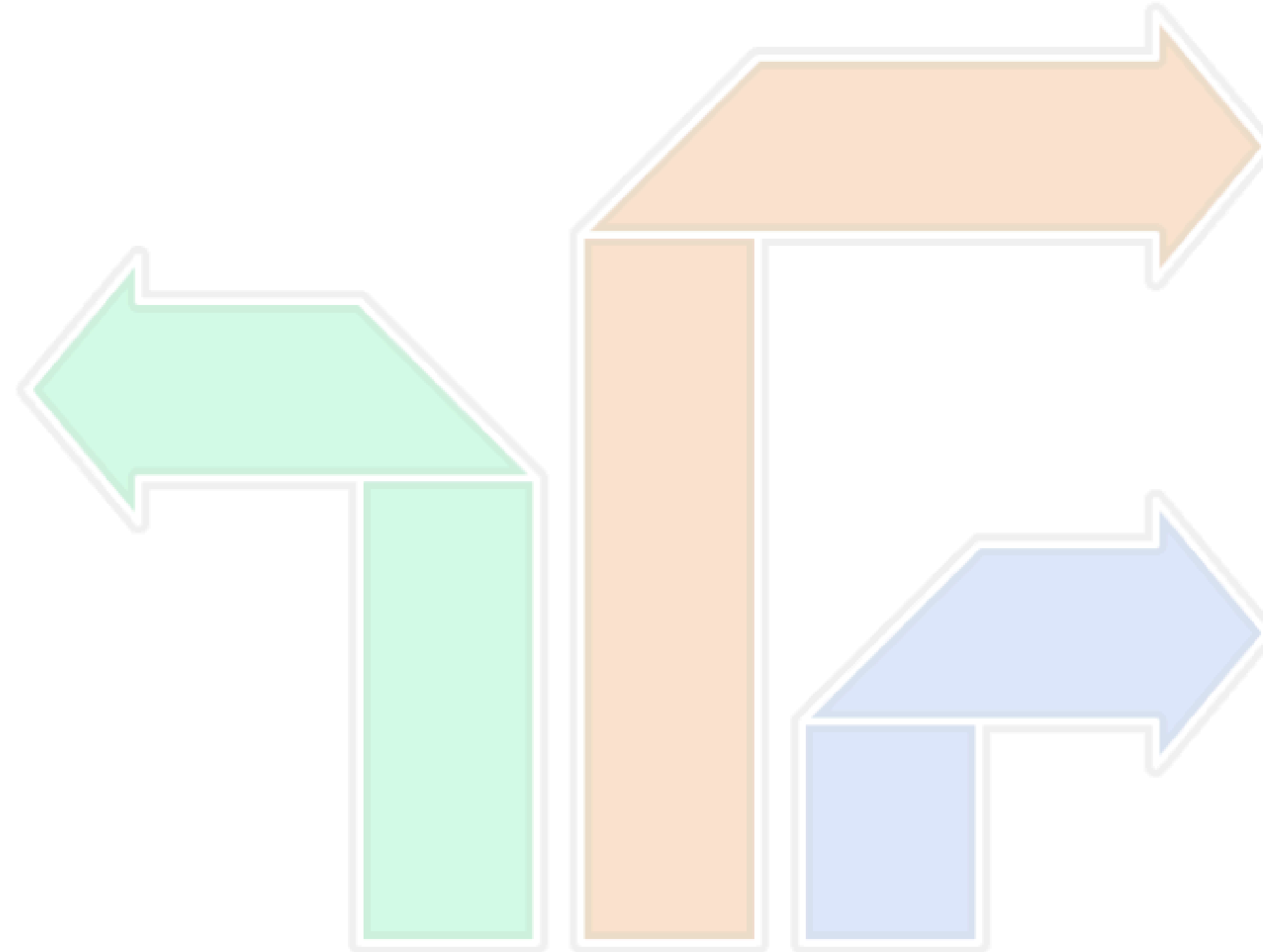
Output



Summary

`.apply()`

Executes the function with a specified 'this' value and an array of arguments.



`.bind()`

Creates a new function with a specified 'this' value and arguments, without immediate execution.

`.call()`

Executes the function with a specified 'this' value and arguments.

Conclusion



Global Object

When 'this' refers to the global object by default



Parent Object

When 'this' refers to the parent object in a function call



New Object

When 'this' refers to a newly created object with 'new'

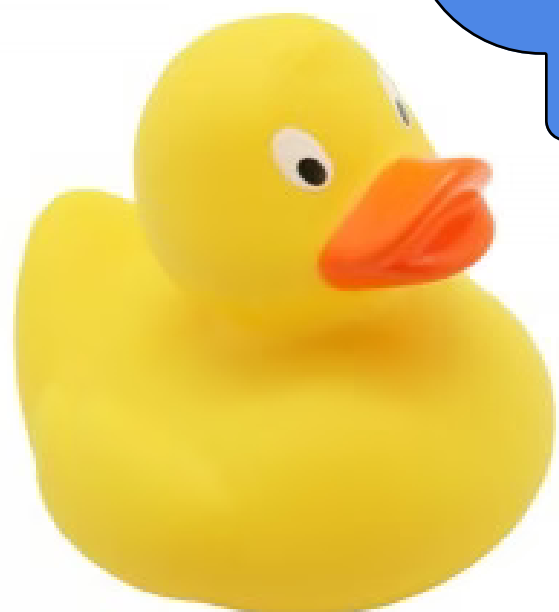


Call/Apply

When 'this' refers to the first argument of 'call' or 'apply'

**Do not ask yourself:
Where is this taken from?**

**Ask yourself:
How is the function invoked?**



Thank you for attention!